

11. Claude Code × Next.js バイブコーディング完全ガイド

Claude Codeを使ってNext.jsアプリをバイブコーディング（自然言語で指示を出しながらAIと協調開発）するための、実践的な作業手順書です。Claude Codeの全機能（Settings、Hooks、Skills、Rules、MCP、Permissions、Plan Mode、Worktrees、Subagents、Memory、Keybindings）を網羅し、各フェーズで最適な機能を活用する方法を解説します。

この章は完全な初学者向けです。「AIエージェントってそもそも何？」というところから始めて、本格的な開発フローまで一気に通貫で解説します。読みながらわからない用語が出てきたら、各セクションの冒頭の用語解説に戻って確認してください。

0. 前提知識: AIアシスタントと一緒にコードを書くて何？

0.0 まず押さえておきたい言葉たち

ここから出てくる言葉を、最初にまとめて整理しておきます。雰囲気だけでもつかんでおくと、以降の説明がぐっと読みやすくなります。

- **AI（人工知能）**：コンピュータが「人間っぽい判断」をしてくれる技術の総称。
- **LLM（Large Language Model、大規模言語モデル）**：大量の文章を学習した、自然言語を扱うAIの一種。Claude や ChatGPT の中身がこれです。
- **AIエージェント**：LLMに「ファイルを読む」「コマンドを実行する」などの行動能力（ツール）を持たせ、目的に向かって自律的に動けるようにした仕組み。Claude Code はまさにこのAIエージェントです。
- **コンテキストウィンドウ**：AIが一度に「読んで覚えていられる」文字数の上限。会話履歴・読み込んだファイル・指示文の全部がここに収まる必要があります。日本語だとざっくり数十万字ぶん。
- **トークン**：AIが文字を扱う最小単位。日本語1文字でだいたい1～2トークン。コンテキストウィンドウの上限はトークン数で決まります。
- **プロンプト**：AIに渡す指示文のこと。「～してください」と書くテキストそのもの。
- **プロンプトエンジニアリング**：AIから良い答えを引き出すために、プロンプトの書き方を工夫する技術。指示を具体的にする・例を見せる・ステップを区切る、などのコツがあります。
- **ハルシネーション**：AIがもっともらしいウソをつく現象。実在しない関数名を呼んだり、間違ったAPIを使ったりすることがあるため、必ず人間が検証する必要があります。
- **ツール呼び出し（tool use / function calling）**：AIエージェントが「ファイル読む」「コマンド打つ」などの行動を起こすときの仕組み。Claude Code では **Read** **Edit** **Bash** などのツールがあらかじめ用意されています。
- **コードレビュー**：書いたコードを他の人（または別のAI）が読み、バグ・規約違反・改善点を指摘する作業。バイブコーディングでは「自分がレビュアー」になります。

- **ペアプロ（ペアプログラミング）**：2人で1台のパソコンに向かい、1人がコードを書き、もう1人が横で見ながら助言する開発スタイル。バイブコーディングは「相手がAI」のペアプロだと考えると分かりやすいです。

これらの用語は本章で何度も出てくるので、いったん流し読みして OK です。

0.1 「バイブコーディング」とは

バイブコーディング（vibe coding）とは、「AIに自然言語で指示しながら、AIが書いたコードを確認・修正していく開発スタイル」のことです。**feel like coding**（コードを書いている雰囲気を味わう）のニュアンスで、自分はざっくりした方針を伝え、細部はAIに任せる感じです。

従来のプログラミングは「手で1行ずつコードを打つ」のが当たり前でしたが、バイブコーディングでは「日本語で要件を伝える → AIが実装する → 自分はそれをレビュー・指示」というサイクルになります。前述のペアプロのうち、相方がAIになったイメージです。

▼ 従来の開発スタイルとの違い:

観点	従来	バイブコーディング
コードを書く主体	自分	主にAI（自分はレビュー）
学習方法	ドキュメントを読む	AIに「これは何？」と聞く
エラー対処	Google検索	エラーをAIに貼って質問
実装速度	遅い〜中	非常に速い
コード理解度	高い	油断すると低くなる

重要な注意: バイブコーディングはコードの「品質保証は自分の責任」です。AIは間違いも書きます（前述のハルシネーション）。生成されたコードを必ず読んで理解する習慣を捨てないでください。

0.1.1 なぜAIに「お願い」だけでなく「文脈」と「制約」を渡すと結果が良くなるのか

これがバイブコーディングの肝です。LLM は確率的に「次に来そうな文字」を選ぶ仕組みなので、入力に含まれる情報が多いほど、より正確で意図に近い答えを出せます。

たとえば「ボタンを作って」と言うだけだと、AIは「どんなボタン？色は？クリックしたら何が起きる？フレームワークは何？」を全部勝手に推測（つまり当てずっぽう）で埋めます。逆に「タスク一覧画面の右上に、Tailwind の bg-blue-600 でスタイリングした追加ボタン。クリックで TaskFormModal を開く。shadcn/ui の Button を使う」と書けば、推測の余地がなくなり、意図どおりのコードが返ってきます。

このため本章では「仕様」「制約」「参考ファイル」「禁止事項」を丁寧にプロンプトに書き込む方法を繰り返し紹介します。

0.2 Claude Code でできることのイメージ

ターミナルで Claude Code を起動して、以下のような自然な指示が通ります。

▼ ユーザーの入力例:

```
書籍管理アプリの新規登録ページを実装して。  
- /books/new でアクセスできる  
- title, author, status をフォームに含める  
- 送信時に Supabase の books テーブルに insert する  
- 完了したら一覧ページにリダイレクト
```

このように、URL・必要なフィールド・処理内容・遷移先まで明示してあると、AI はほぼ迷わずに作業できます。

▼ Claude Codeの動作（イメージ）：

```
[Claude] 了解しました。以下の手順で実装します。  
1. app/books/new/page.tsx を作成  
2. components/BookForm.tsx を作成  
3. Server Action createBook を追加  
  
[Edit] app/books/new/page.tsx を作成中...  
[Edit] components/BookForm.tsx を作成中...  
[Edit] app/actions/books.ts を更新中...  
  
実装が完了しました。/books/new にアクセスして動作確認してください。
```

[Edit] の部分は、Claude Code が **Edit** ツール（ファイルを変更するツール）を呼んでいることを示しています。Claude Code は内部に複数のツールを持っており、状況に応じて使い分けます。主要なツールは次のとおりです。

ツール名	役割
Read	ファイルの中身を読む
Edit	既存ファイルの一部を書き換える（差分編集）
Write	ファイルを新規作成、または全体を上書き
Grep	ファイル内容を正規表現で検索（中身検索）
Glob	ファイル名パターン（ **/*.tsx など）で検索
Bash	シェルコマンドを実行（ npm install など）
Agent / Task	サブエージェントを起動して別タスクに委任

0.3 まず触ってみる: 最小コマンド集

やりたいこと	入力するコマンド/操作
Claude Code を起動	プロジェクトフォルダで <code>claude</code>
新しい会話を始める	<code>/clear</code>
プランモード切替	<code>Shift+Tab</code> (読み取り専用で計画だけ立てる)
設定画面を開く	<code>/config</code>
ヘルプ	<code>/help</code>
終了	<code>/exit</code> または <code>Ctrl+D</code>

`/help` や `/clear` のように「スラッシュ `/` で始まるコマンド」を **スラッシュコマンド** と呼びます。チャット欄にメッセージを送るのではなく、Claude Code 自体に対する操作命令を表します。スラッシュコマンドは Tab キーで補完できます。

0.4 安全に使うための基本ルール

1. 重要な変更は Git コミット直後に依頼する — いつでも `git reset` で戻せる状態にしておく。AIに任せる前に必ず `git status` でクリーンな状態かを確認しましょう。
2. 生成コードを必ず読む — 「動いたからOK」ではなく「理解したからOK」を目標に。読まずにマージしてしまうと、自分のリポジトリなのに何が起きているか分からなくなります。
3. `rm -rf` や `git push --force` などの破壊的コマンドはユーザー確認を要求する設定にする — `.claude/settings.json` で制御可能（後の章で説明）。
4. API キーや秘密情報を AI に直接見せない — `.env.local` や `git-ignore` を活用。AIに `.env` の中身を「読んで」と頼んでも答えてくれますが、その内容が会話履歴に残るため漏えいリスクが上がります。
5. `git diff` や `pwd` で自分の状態を確認する — 「いま自分はどのブランチにいて」「どのファイルが変更中で」「どのディレクトリにいるか」を把握しないまま AI に指示を出すと、思わぬところに変更が入ってしまいます。
6. AIに自動でコミット・プッシュさせない — コミット内容は人間が `git diff` で確認してから push しましょう。AI に直接 `git push` を実行させるのは原則禁止です。

目次

1. Claude Code の全体像と主要機能
2. Phase 0：環境構築と安全設定
3. Phase 1：プロジェクト初期化
4. Phase 2：要件定義（Plan Mode活用）
5. Phase 3：CLAUDE.md の作成と配置戦略
6. Phase 4：Rules の構成

- 7. Phase 5 : Skills の作成と活用
 - 8. Phase 6 : Subagents (カスタムエージェント) の構成
 - 9. Phase 7 : 開発フェーズ — 機能実装の進め方
 - 10. Phase 8 : テスト・品質管理
 - 11. Phase 9 : デプロイと最終調整
 - 12. Context管理とMemoryシステム
 - 13. キーボードショートカットのカスタマイズ
 - 14. Worktrees (並列開発)
 - 15. CLIフラグとオプション一覧
 - 16. トラブルシューティング
 - 17. ベストプラクティス総まとめ
 - 18. 付録 : 設定ファイルパス早見表
-

1. Claude Code の全体像と主要機能

1-1. Claude Code とは

Claude CodeはAnthropicが提供する公式CLIツールで、ターミナル上でClaudeと対話しながらソフトウェア開発を行えます。CLI、デスクトップアプリ (Mac/Windows)、Webアプリ (claude.ai/code)、IDE拡張 (VS Code、JetBrains) で利用可能です。

「CLI」とは Command Line Interface の略で、ターミナル (黒い画面) から文字入力で操作するタイプのアプリのことです。GUI (マウス操作のアプリ) と比べてキーボード中心で素早く操作でき、スクリプトとも連携しやすいのが特徴です。

1-2. 設定ファイルの階層構造

Claude Codeは4層のスコプで設定を管理します。**最も具体的なスコプが優先**されます。「スコプ」とは設定が効く範囲のこと。「自分のマシン全体」よりも「いまのプロジェクトだけ」のほうがより具体的、という意味です。

優先度: 高 ←————→ 低 (より具体的なスコープが優先)

Managed (企業管理者向け)

場所: `C:\Program Files\ClaudeCode\settings.json`

対象: マシン全ユーザー、IT 管理者が制御

User (個人設定)

場所: `~/.claude/settings.json`

対象: 自分の全プロジェクト共通

Project (プロジェクト設定)

場所: `.claude/settings.json`

対象: このリポジトリの全メンバー (Git 管理可能)

Local (ローカル設定)

場所: `.claude/settings.local.json`

対象: 自分のこのプロジェクトのみ (gitignore 推奨)

~ (チルダ) はホームディレクトリの略記号で、Windows では `C:\Users\<ユーザー名>` を指します。`.claude` のように先頭にドットが付くフォルダは「隠しフォルダ」で、エクスプローラの設定によっては見えません。

1-3. 主要機能一覧

機能	説明	設定場所
Settings	権限・モデル・挙動の制御	<code>settings.json</code>
CLAUDE.md	プロジェクト固有の指示書	プロジェクトルート等
Rules	パス別の自動適用ルール	<code>.claude/rules/*.md</code>
Skills	再利用可能なプロンプトテンプレート	<code>.claude/skills/*/SKILL.md</code>
Subagents	カスタムAIエージェント定義	<code>.claude/agents/*.md</code>
Hooks	イベント駆動の自動化	<code>settings.json</code> 内
MCP	外部サービス連携	<code>.mcp.json</code>
Permissions	コマンド実行の許可/拒否	<code>settings.json</code> 内
Memory	会話を跨ぐ記憶	<code>~/.claude/projects/*/memory/</code>
Keybindings	キーボードショートカット	<code>~/.claude/keybindings.json</code>
Worktrees	Git worktreeで並列開発	CLI引数
Plan Mode	読み取り専用の設計モード	<code>Shift+Tab</code> で切り替え

それぞれの詳細は後続の Phase で順番に登場します。「いま全部理解しなくてOK」、表は索引代わりに使ってください。

2. Phase 0：環境構築と安全設定

2-1. Claude Code のインストール

```
# Node.js 18以上が必要 (npm を使うため、まず Node.js を入れておく)
# Claude Code 本体を npm のグローバル領域にインストールする
# -g は global の意味で、どのフォルダからでも claude コマンドが使えるようになる
# @anthropic-ai/claude-code は npm 上のパッケージ名 (スコープ名@パッケージ名)
npm install -g @anthropic-ai/claude-code

# インストール確認: バージョンが出れば成功
# --version は「現在のバージョン情報だけ出して終了」を意味する一般的なオプション
claude --version

# APIキーの設定 (初回のみ)
# 引数なしで claude を実行すると対話モードが起動する
# 起動時に Anthropic の API キーまたはアカウント認証を求められる
claude
```

Node.js はJavaScriptをパソコン上で動かすための実行環境です。バージョン 18 以上が必要で、なければ <https://nodejs.org> からインストールしてください。

2-2. 危険コマンドの禁止設定 (Permissions)

これが最も重要な初期設定です。必ず開発を始める前に設定してください。AIエージェントは賢いとはいえ、勘違いやハルシネーションで「**rm -rf .**」のような危険コマンドを提案することがあります。事前に「これは絶対に実行しない」と Claude Code 側でブロックしておくのが安全策です。

Claude Codeへのプロンプト：


```
# /update-config は設定ファイルを編集するための組み込みスキル
# このスキルを呼び出すと Claude が settings.json の編集をサポートしてくれる
/update-config
```

Claude Codeの ~/.claude/settings.json を設定してください。
以下のbashコマンドをdenyリストに追加してください：

1. ファイル削除系：
 - rm -rf
 - rm -r
 - del /s (Windows)
 - rmdir /s (Windows)
2. Git破壊系：
 - git push --force
 - git reset --hard
 - git clean -fd
 - git checkout . (全ファイル復元)
 - git branch -D (強制ブランチ削除)
3. データベース破壊系：
 - DROP TABLE
 - DROP DATABASE
 - TRUNCATE
4. システム系：
 - sudo
 - chmod 777

また以下のallowリストも設定してください：

- Read (全ファイル読み取り)
- Bash(npm run *)
- Bash(npx *)
- Bash(git status*)
- Bash(git add *)
- Bash(git commit *)
- Bash(git log*)
- Bash(git diff*)
- Bash(git branch*)

手動で設定する場合の完全な `~/.claude/settings.json`：

```

{
  "permissions": {
    "defaultMode": "default",
    "allow": [
      "Read",
      "Glob",
      "Grep",
      "Bash(npm run *)",
      "Bash(npm install *)",
      "Bash(npm uninstall *)",
      "Bash(npx *)",
      "Bash(node *)",
      "Bash(git status*)",
      "Bash(git add *)",
      "Bash(git commit *)",
      "Bash(git log*)",
      "Bash(git diff*)",
      "Bash(git branch*)",
      "Bash(git checkout -b *)",
      "Bash(git switch *)",
      "Bash(git stash*)",
      "Bash(git merge *)",
      "Bash(git pull*)",
      "Bash(git fetch*)",
      "Bash(ls *)",
      "Bash(cat *)",
      "Bash(mkdir *)",
      "Bash(cp *)",
      "Bash(mv *)",
      "Bash(echo *)",
      "Bash(cd *)",
      "Bash(pwd)",
      "Bash(which *)",
      "Bash(curl *)"
    ],
    "deny": [
      "Bash(rm -rf *)",
      "Bash(rm -r *)",
      "Bash(rm *)",
      "Bash(del /s *)",
      "Bash(rmdir /s *)",
      "Bash(git push --force*)",
      "Bash(git push -f *)",
      "Bash(git reset --hard*)",
      "Bash(git clean -fd*)",
      "Bash(git clean -f*)",
      "Bash(git checkout -- .)",
      "Bash(git branch -D *)",
      "Bash(*DROP TABLE*)"
    ]
  }
}

```

```

    "Bash(*DROP DATABASE*)",
    "Bash(*TRUNCATE*)",
    "Bash(sudo *)",
    "Bash(chmod 777 *)",
    "Bash(shutdown*)",
    "Bash(reboot*)",
    "Edit(/.git/**)",
    "Edit(/.env)",
    "Edit(/.env.local)"
  ]
}
}

```

各キーの意味は次のとおりです。

- **permissions** : 権限制御のルートキー。許可・拒否設定はすべてこの下にぶら下がる。
- **defaultMode** : 普段使うパーミッションモード。 **default** は「毎回確認」モード。
- **allow** : 自動的に許可するツール・コマンドのリスト。マッチしたら確認なしで実行される。
- **deny** : 絶対に実行させないツール・コマンドのリスト。マッチしたら確認すら出ずブロック。
- **Bash(...)** : Bash ツールの引数パターン。 ***** はワイルドカードで任意の文字列にマッチ。
- **Edit(/.env)** : Edit ツールで **/.env** を編集しようとするのをブロック。秘密情報の保護。

Permission ルールの優先順位: Deny > Ask > Allow（最初にマッチしたルールが適用）。つまり「**deny** に書いた条件は、**allow** に同じものがあったとしても deny が勝つ」。

ワイルドカード * : パターンマッチで広範囲にブロック可能。 **Bash(rm -rf *)** は **rm -rf /** も **rm -rf .** もブロックします。

2-3. Permission Mode の種類と使い分け

Claude Codeには6つのパーミッションモードがあります。セッション中に **Shift+Tab** で切り替え可能です。

モード	コマンド	説明	使う場面
default	<code>claude</code>	各操作ごとに確認を求める	通常の開発
acceptEdits	<code>claude --permission-mode acceptEdits</code>	ファイル編集は自動承認	信頼できるタスク
plan	<code>claude --permission-mode plan</code>	読み取り専用（編集不可）	設計・調査フェーズ
auto	<code>claude --permission-mode auto</code>	バックグラウンドで安全性チェック	CI/CD連携
dontAsk	<code>claude --permission-mode dontAsk</code>	事前許可済みツールのみ	自動化パイプライン
bypassPermissions	<code>claude --dangerously-skip-permissions</code>	チェックなし（危険）	使用非推奨

`--permission-mode` は「パーミッションモードを指定して起動する」フラグです。`--` で始まるオプションは「ロングオプション」と呼ばれ、後ろに値を渡せます。

デフォルトモードを設定する場合：

```
{
  "permissions": {
    "defaultMode": "default"
  }
}
```

- `permissions` : 設定のグループ名。
- `defaultMode` : 何も指定せず `claude` と打ったときに使うモード。

2-4. MCP サーバーの設定

MCP（Model Context Protocol）により、Claude Codeに外部サービスとの連携能力を追加できます。MCP は「AIに外部ツールを足すための共通プロトコル」で、これを使うと GitHub の Issue を読んだり Supabase のテーブルを操作したり、といった外部連携が可能になります。

MCP設定ファイルの階層

優先度: 高 ← → 低

<code>.claude/.mcp.local.json</code>	← 個人用 (gitignore)
<code>.claude/.mcp.json</code>	← プロジェクト共有 (Git管理)
<code>~/.claude/.mcp.json</code>	← 全プロジェクト共通

Claude Codeへのプロンプト:

以下のMCPサーバーを設定してください。

1. GitHub MCP
 - GitHubのIssue/PR操作に使用
 - 個人アクセストークンが必要
2. Supabase MCP
 - データベース操作に使用
 - Supabaseアクセストークンが必要
3. Playwright MCP (オプション)
 - ブラウザテスト自動化に使用

ユーザーレベル (`~/.claude/.mcp.json`) に設定してください。
トークンは環境変数から読み込む形式にしてください。

手動設定の場合 — `~/.claude/.mcp.json` :

```
{
  "mcpServers": {
    "github": {
      "type": "stdio",
      "command": "npx",
      "args": ["-y", "@modelcontextprotocol/server-github"],
      "env": {
        "GITHUB_PERSONAL_ACCESS_TOKEN": "$GITHUB_TOKEN"
      },
      "allowedEnvVars": ["GITHUB_TOKEN"]
    },
    "supabase": {
      "type": "stdio",
      "command": "npx",
      "args": ["-y", "@supabase/mcp-server"],
      "env": {
        "SUPABASE_ACCESS_TOKEN": "$SUPABASE_TOKEN"
      },
      "allowedEnvVars": ["SUPABASE_TOKEN"]
    },
    "playwright": {
      "type": "stdio",
      "command": "npx",
      "args": ["-y", "@anthropic-ai/mcp-server-playwright"]
    }
  }
}
```

各キーの意味は以下のとおりです。

- **mcpServers** : MCP サーバー定義をまとめるルートキー。
- **"github"** などのキー: サーバーの識別名。Claude 側で参照するためのラベル。
- **type** : 接続方式。 **stdio** (標準入出力) / **http** / **sse** から選ぶ。
- **command** : サーバーを起動するコマンド。 **npx** は npm パッケージを直接実行するツール。
- **args** : 上記コマンドに渡す引数の配列。 **-y** は npx の「確認なしで進める」フラグ。
- **env** : サーバーに渡す環境変数。 **\$GITHUB_TOKEN** でシェルの環境変数を参照。
- **allowedEnvVars** : 参照を許可する環境変数のホワイトリスト。セキュリティのため明示が必要。

MCP接続タイプの違い

タイプ	形式	用途
stdio	ローカルプロセス	npm パッケージ型のMCPサーバー
http	HTTP接続	リモートAPIサービス
sse	Server-Sent Events	ストリーミング対応サービス

HTTP型の例（Sentry連携）：

```
{
  "mcpServers": {
    "sentry": {
      "type": "sse",
      "url": "https://mcp.sentry.io/stream"
    }
  }
}
```

- **type: "sse"**：サーバーから送られてくるイベントを受け取り続けるタイプ。
- **url**：接続先 URL。

OAuth認証付きの例：

```
{
  "mcpServers": {
    "github-oauth": {
      "type": "http",
      "url": "https://mcp.github.io",
      "auth": {
        "type": "oauth",
        "clientId": "abc123",
        "clientSecret": "$GITHUB_CLIENT_SECRET",
        "redirectPort": 8080
      },
      "allowedEnvVars": ["GITHUB_CLIENT_SECRET"]
    }
  }
}
```

- **auth**：認証情報のグループ。
- **type: "oauth"**：OAuth2.0 認証フローを使用。
- **clientId** / **clientSecret**：OAuth アプリケーションの ID と秘密鍵。
- **redirectPort**：認証完了後にリダイレクトを受け取るローカルポート。

MCP内で使える環境変数

```
{
  "args": ["${CLAUDE_PROJECT_DIR}/scripts/server.js"],
  "env": {
    "SESSION": "${CLAUDE_SESSION_ID}",
    "PROJECT": "${CLAUDE_PROJECT_DIR}"
  }
}
```

`${VAR}` の形式は Claude Code が用意した特殊変数の展開構文です。シェル変数とは異なり、Claude Code 側で値が差し込まれます。

変数	説明
<code>\${CLAUDE_SESSION_ID}</code>	現在のセッションID
<code>\${CLAUDE_PROJECT_DIR}</code>	プロジェクトルートパス
<code>\${CLAUDE_TOOLS_DIR}</code>	ツールディレクトリ
<code>\$VAR_NAME</code>	<code>allowedEnvVars</code> に登録した環境変数

2-5. Hooks の設定（イベント駆動自動化）

Hooksは特定のイベントが発生した時に自動でコマンドを実行する仕組みです。コード品質の自動維持に不可欠です。たとえば「ファイル編集のたびに自動で Prettier をかける」「コミット前に lint を走らせる」など、機械的な作業を肩代わりさせられます。

全Hookイベント一覧

イベント	発火タイミング	主な用途
SessionStart	セッション開始/再開時	コンテキスト注入、環境設定
UserPromptSubmit	ユーザーがプロンプト送信時	入力バリデーション
PreToolUse	ツール実行前	危険操作のブロック、入力の書き換え
PostToolUse	ツール実行成功後	コードフォーマット、リント
PostToolUseFailure	ツール実行失敗後	エラー処理
Notification	通知発火時	デスクトップ通知連携
Stop	Claudeの応答完了時	最終チェック
PreCompact	コンテキスト圧縮前	重要情報の退避
PostCompact	コンテキスト圧縮後	コンテキスト再注入
SessionEnd	セッション終了時	クリーンアップ

Hook設定の構造

```
{
  "hooks": {
    "イベント名": [
      {
        "matcher": "マッチパターン（ツール名やイベント種別）",
        "hooks": [
          {
            "type": "command",
            "command": "実行するシェルコマンド",
            "timeout": 600
          }
        ]
      }
    ]
  }
}
```

- `hooks`: フック定義をまとめるルートキー。
- `"イベント名"`: 上の表のイベント名（例 `PostToolUse`）。
- `matcher`: そのイベント内でさらに絞り込むためのパターン。たとえば `Edit|Write` で Edit と Write ツールだけにマッチ。
- 内側の `hooks` 配列: 実行する処理のリスト。

- `type: "command"` : シェルコマンドを実行する種別。
- `command` : 実行する具体的なコマンド文字列。
- `timeout` : 何秒で打ち切るか（秒単位）。重い処理を放置しないための保険。

Hookの終了コードの意味

終了コード	意味	動作
0	成功	JSON出力があればパース
2	ブロッキングエラー	stderrがClaudeへフィードバックされる
その他	非ブロッキングエラー	トランスクリプトに記録のみ

「終了コード」とはコマンドの結果を表す整数で、0 が成功、それ以外は何らかのエラー、というのが UNIX 系の慣習です。

Claude Codeへのプロンプト：

```
/update-config
```

Claude Codeのhooksを以下のように設定してください。
settings.jsonに追記してください。

1. PostToolUse: ファイル編集(Edit/Write)後にPrettierで自動フォーマット
2. PostToolUse: ファイル編集後にESLintで自動修正
3. PreToolUse: .envファイルへの書き込みをブロック
4. SessionStart: セッション開始時にリマインダーを表示
「テストを書いてからコミットすること」
5. Notification: デスクトップ通知 (Windows用PowerShell)
6. PostCompact: コンテキスト圧縮後にCLAUDE.mdの重要ルールを再注入

手動設定の完全例（`~/.claude/settings.json` に追記）：

```

{
  "hooks": {
    "PostToolUse": [
      {
        "matcher": "Edit|Write",
        "hooks": [
          {
            "type": "command",
            "command": "FILE=$(echo $CLAUDE_TOOL_INPUT | jq -r '.file_path // empty')
&& [ -n \"$FILE\" ] && npx prettier --write \"$FILE\" 2>/dev/null || true",
            "timeout": 30
          }
        ]
      },
      {
        "matcher": "Edit|Write",
        "hooks": [
          {
            "type": "command",
            "command": "FILE=$(echo $CLAUDE_TOOL_INPUT | jq -r '.file_path // empty')
&& [ -n \"$FILE\" ] && [[ \"$FILE\" == *.ts || \"$FILE\" == *.tsx ]] && npx eslint --
fix \"$FILE\" 2>/dev/null || true",
            "timeout": 30
          }
        ]
      }
    ],
    "PreToolUse": [
      {
        "matcher": "Edit|Write",
        "hooks": [
          {
            "type": "command",
            "command": "INPUT=$(cat) && FILE_PATH=$(echo \"$INPUT\" | jq -r
'.tool_input.file_path // empty') && if [[ \"$FILE_PATH\" == *.env* ]]; then echo
'BLOCKED: .envファイルの編集は禁止されています' >&2; exit 2; fi; exit 0"
          }
        ]
      }
    ],
    "SessionStart": [
      {
        "matcher": "",
        "hooks": [
          {
            "type": "command",
            "command": "echo '[リマインダー] テストを書いてからコミットしてください。CLAUDE.md
の規約を遵守してください。'"
          }
        ]
      }
    ]
  }
}

```

```

    ]
  },
  ],
  "PostCompact": [
    {
      "matcher": "",
      "hooks": [
        {
          "type": "command",
          "command": "echo '[コンテキスト再注入] 重要: any型禁止、Server Componentをデフォルト使用、コミットは日本語プレフィックス付き'"
        }
      ]
    }
  ],
  ],
  "Notification": [
    {
      "matcher": "",
      "hooks": [
        {
          "type": "command",
          "command": "powershell.exe -Command \"[System.Reflection.Assembly]::LoadWithPartialName('System.Windows.Forms');[System.Windows.Forms.MessageBox]::Show('Claude Codeの確認が必要です', 'Claude Code')\""
        }
      ]
    }
  ],
  ],
  }
}

```

長いのでポイントだけ解説します。

- `"matcher": "Edit|Write"`: 正規表現で「Edit または Write」を意味し、その両ツールの実行後に発火。
- `FILE=$(echo $CLAUDE_TOOL_INPUT | jq -r '.file_path // empty')`: ツール入力 JSON から `file_path` フィールドを取り出す。jq は JSON 操作の定番ツール。
- `[ -n "$FILE" ] && ... : $FILE` が空でないときだけ次のコマンドを実行（&& は「左が成功したら右を実行」）。
- `npm run prettier --write "$FILE"`: Prettier で対象ファイルをフォーマット。--write は実ファイルへの書き戻し。
- `2>/dev/null || true`: エラーを捨てて常に成功扱い。Hook 全体は壊さない設計。
- `exit 2`: Claude にブロックを伝える終了コード。echo ... >&2 で標準エラー出力にメッセージを出す。
- `powershell.exe -Command ...`: Windows の PowerShell でメッセージボックスを表示。

高度なHook例：保護ファイルのブロックスクリプト

プロジェクト内にスクリプトを配置して、より複雑なロジックを実行できます。

`.claude/hooks/protect-files.sh` :

```
#!/bin/bash
# 上の行は shebang (シバン)。このスクリプトを bash で実行するよう指示
# 標準入力 (Claude が JSON で渡してくる情報) を変数 INPUT に読み込む
INPUT=$(cat)
# jq でツール入力の file_path フィールドを取り出す。値がなければ空文字列
FILE_PATH=$(echo "$INPUT" | jq -r '.tool_input.file_path // empty')

# 保護対象ファイルリスト (配列)
PROTECTED=(
    ".env"
    ".env.local"
    ".env.production"
    "package-lock.json"
    ".git/"
    "next.config.js"
)

# 配列の各パターンに対してループ
for pattern in "${PROTECTED[@]}; do
    # FILE_PATH に保護対象パターンが含まれていればブロック
    if [[ "$FILE_PATH" == *"$pattern"* ]]; then
        # 標準エラー出力にメッセージを書き、終了コード 2 でブロック
        echo "BLOCKED: $FILE_PATH は保護されたファイルです。手動で編集してください。" >&2
        exit 2
    fi
done

# 何にもマッチしなければ正常終了
exit 0
```

```
{
  "hooks": {
    "PreToolUse": [
      {
        "matcher": "Edit|Write",
        "hooks": [
          {
            "type": "command",
            "command": "bash \"$CLAUDE_PROJECT_DIR/.claude/hooks/protect-files.sh\""
          }
        ]
      }
    ]
  }
}
```

- `"$CLAUDE_PROJECT_DIR/.claude/hooks/protect-files.sh"`: プロジェクトルート配下のスクリプトを実行。複雑なロジックはスクリプトに切り出すと管理しやすい。

3. Phase 1：プロジェクト初期化

3-1. Next.js プロジェクトの作成

Claude Codeへのプロンプト：

Next.jsの新規プロジェクトを作成してください。以下の構成をお願いします：

- プロジェクト名: my-app
- TypeScript: 使用する
- App Router: 使用する
- Tailwind CSS v4: 使用する
- ESLint: 使用する
- src/ ディレクトリ: 使用する
- import alias: @/*
- パッケージマネージャ: npm

create-next-app 実行後、追加で以下もインストール：

- shadcn/ui 初期化 (New Yorkスタイル)
- zod (バリデーション)
- react-hook-form + @hookform/resolvers
- zustand (状態管理)
- date-fns (日付処理)
- lucide-react (アイコン)

インストールが完了したら、動作確認のため `npm run dev` を実行してビルドエラーがないことを確認してください。

まだ機能のコードは書かないでください。

「まだ機能のコードは書かないでください」と書くことで、AIが暴走して未承認のページや機能を追加するのを防いでいます。

3-2. Git リポジトリの初期化

Claude Codeへのプロンプト：

gitリポジトリを初期化して、以下を設定してください：

1. `.gitignore` に以下を追加
 - `.env*` (`.env.example`は除外)
 - `.claude/settings.local.json`
 - `.claude/.mcp.local.json`
 - `node_modules/`
 - `.next/`
 - `*.log`
2. `.env.example` を作成（値は空で、変数名のみ）
 - `NEXT_PUBLIC_SUPABASE_URL=`
 - `NEXT_PUBLIC_SUPABASE_ANON_KEY=`
 - `SUPABASE_SERVICE_ROLE_KEY=`
3. 初回コミット：「feat: Next.jsプロジェクト初期構築」

`.env`ファイルが含まれていないことを確認してからコミットしてください。

`.gitignore` は「Git の追跡対象から除外するファイル」を指定するファイルです。`.env`（秘密情報）や `node_modules`（巨大）は必ず除外しないと事故ります。

3-3. Claude Code 用ディレクトリ構造の作成

Claude Codeへのプロンプト：

プロジェクト内に以下のClaude Code用ディレクトリ構造を作成してください。
まだ中身のファイルは作らなくて結構です。

```
.claude/
├─ settings.json           # プロジェクト共有設定
├─ settings.local.json    # 個人設定 (gitignore)
├─ .mcp.json              # MCP設定 (プロジェクト共有)
├─ .mcp.local.json        # MCP設定 (個人用、gitignore)
├─ rules/                 # パス別自動適用ルール
│   └─ components.md
│   └─ api-routes.md
│   └─ database.md
│   └─ testing.md
├─ skills/                # カスタムSkills
│   └─ review/
│       └─ SKILL.md
│   └─ create-component/
│       └─ SKILL.md
│   └─ create-api/
│       └─ SKILL.md
├─ agents/               # カスタムSubagents
│   └─ code-reviewer.md
│   └─ test-writer.md
└─ hooks/               # Hookスクリプト
    └─ protect-files.sh
```

ここで作るディレクトリは後の Phase で1つずつ埋めていきます。先に「箱」だけ作っておくと、後の設定作業が見通しよく進みます。

4. Phase 2：要件定義（Plan Mode活用）

4-1. Plan Mode とは

Plan Modeは読み取り専用のモードで、Claudeはコードを読んで分析はできますが、ファイルの作成・編集・コマンド実行はできません。設計フェーズに最適です。

要件が定まる前に AI に勝手にコードを書かれてしまうと、後戻りが大変です。「最初は計画だけ立てる」というのを Plan Mode で強制できます。

Plan Mode の起動方法

```
# 起動時にPlan Modeで開始
# --permission-mode plan で読み取り専用モードに固定
claude --permission-mode plan

# セッション中に切り替え
# Shift+Tab を押す → "plan" を選択
```

Plan Mode の活用フロー

```
Plan Mode (設計)  → 方針合意  →  Default Mode (実装)

|                  |
| ・要件整理       | ・コード生成
| ・画面設計       | ・ファイル作成
| ・DB設計         | ・テスト実行
| ・API設計        |
| ・実装方針の提案 |
↓                  ↓
承認オプション:
├─ Auto Mode (安全チェック付き自動実行)
├─ Accept Edits Mode (ファイル編集は自動承認)
├─ Default Mode (各操作ごとに確認)
└─ Keep Planning (さらに設計を続行)
```

4-2. アプリ全体像の伝え方

Plan Modeを起動してから以下を入力：

以下のWebアプリケーションを開発します。
まず要件を整理してドキュメントとして出力してください。

アプリ概要

タスク管理Webアプリ「TaskFlow」

ターゲットユーザー

- 小規模チーム（5-10人）のプロジェクトマネージャー
- 日常的にタスクの割り振りと進捗管理を行う

主要機能（優先度順）

P0（必須）

1. ユーザー認証（メール/パスワード、Google OAuth）
2. プロジェクト作成・管理
3. タスクのCRUD（作成・表示・更新・削除）
4. タスクのステータス管理（未着手/進行中/レビュー/完了）
5. カンバンボード表示

P1（重要）

6. チームメンバーへのタスク割り当て
7. 期限設定とリマインダー通知
8. ダッシュボード（進捗の可視化）
9. タスクのフィルタリング・検索

P2（あれば嬉しい）

10. タスクへのコメント機能
11. ファイル添付
12. アクティビティログ

技術スタック

- フロントエンド: Next.js 15 (App Router), TypeScript, Tailwind CSS v4, shadcn/ui
- バックエンド: Next.js Route Handlers
- データベース: Supabase (PostgreSQL)
- 認証: Supabase Auth
- 状態管理: Zustand
- バリデーション: Zod
- デプロイ: Vercel

非機能要件

- レスポンシブ対応（モバイルファースト）
- ダークモード対応
- ページ読み込み3秒以内（LCP）
- アクセシビリティ（WCAG 2.1 AA準拠）
- 日本語UI

出力してほしいもの

1. 画面一覧（画面名、URL、概要）
2. 全テーブルのER図（テキスト形式）
3. 全テーブルのカラム定義

4. API一覧（メソッド、パス、リクエスト/レスポンス）
5. 実装の推奨順序

コードは絶対に書かないでください。ドキュメントのみ出力してください。

なぜPlan Modeを使うのか？ - コードを書けないため、設計に集中できる - 誤ってファイルが生成されるリスクがゼロ
- 要件の漏れを防げる

4-3. 画面設計の詳細化

Plan Mode で継続：

先ほどの画面一覧を元に、各画面の詳細を設計してください。

各画面について以下を定義：

1. 画面名とURL（ルーティングパス）
2. ページの種類（Server Component / Client Component）
3. 画面レイアウト（ヘッダー、サイドバー、メインエリアの配置）
4. 表示する要素の一覧（UI要素単位で）
5. ユーザーが行える操作（ボタン、フォーム、ドラッグ等）
6. 各操作に対応するAPI呼び出し
7. ローディング状態の表示方法
8. エラー状態の表示方法
9. 空状態（データなし）の表示方法
10. レスポンシブ時の変更点（モバイル/タブレット/デスクトップ）

対象画面：

- ログイン画面（/login）
- 新規登録画面（/register）
- ダッシュボード（/dashboard）
- プロジェクト一覧（/projects）
- プロジェクト詳細（/projects/[id]）
- カンバンボード（/projects/[id]/board）
- タスク詳細（モーダル or /projects/[id]/tasks/[taskId]）
- メンバー管理（/projects/[id]/members）
- 設定画面（/settings）
- プロフィール画面（/settings/profile）

4-4. データベース設計

Plan Mode で継続：

データベースのテーブル設計を詳細に定義してください。

各テーブルについて以下を含めてください：

1. テーブル名（スネークケース、複数形）
2. 全カラム定義
 - カラム名
 - データ型（PostgreSQL）
 - NOT NULL制約
 - デフォルト値
 - UNIQUE制約
 - CHECK制約
3. プライマリキー（UUID使用）
4. 外部キー（リレーション）
5. インデックス（頻繁にWHEREで使うカラム）
6. RLS（Row Level Security）ポリシー
 - SELECT：誰が読めるか
 - INSERT：誰が作成できるか
 - UPDATE：誰が更新できるか
 - DELETE：誰が削除できるか
7. トリガー（updated_at の自動更新など）

共通ルール：

- 全テーブルに `id`（UUID）、`created_at`、`updated_at` を含める
- ソフトデリート（`deleted_at`）を採用
- タイムスタンプは全て `timestampz`（タイムゾーン付き）

テーブル一覧：

- `profiles`（ユーザープロフィール）
- `projects`（プロジェクト）
- `project_members`（プロジェクトメンバー、多対多の中間テーブル）
- `tasks`（タスク）
- `task_comments`（コメント）
- `task_labels`（ラベル、多対多の中間テーブル）
- `labels`（ラベルマスタ）

SupabaseのマイグレーションSQL形式で出力してください。

4-5. 設計ドキュメントの保存

Plan Mode からDefault Modeに切り替えてから：

(Shift+Tab でDefault Modeに切り替え)

先ほどの設計内容をドキュメントとして保存してください。
以下のファイルに分けて作成：

1. docs/requirements.md - 要件定義書
2. docs/screens.md - 画面設計書
3. docs/database.md - データベース設計書
4. docs/api.md - API設計書
5. supabase/migrations/001_initial_schema.sql - 初回マイグレーション

これらのドキュメントは今後の開発中に参照するため、
具体的かつ正確に記述してください。

ここで作るドキュメントは、後の Phase で `@docs/requirements.md` のように `@` 参照で読み込ませて、AI に常に同じ前提を共有させるための「正典」になります。

5. Phase 3：CLAUDE.md の作成と配置戦略

5-1. CLAUDE.md とは

CLAUDE.mdはClaude Codeへの「永続的な指示書」です。セッション開始時に自動的に読み込まれ、Claudeの振る舞いを制御します。

人間で言うなら「新人さんへの引き継ぎマニュアル」に近いです。プロジェクトの命名規約・禁止事項・コミットルールなどを書いておくと、Claude が毎回それを踏まえて作業してくれます。

5-2. 配置場所と優先順位

優先度（全て結合される。上書きではなく追加）：

C:\Program Files\ClaudeCode\CLAUDE.md	← 企業ポリシー（IT管理者）
~/.claude/CLAUDE.md	← 全プロジェクト共通の個人ルール
./CLAUDE.md	← プロジェクトルート（Git共有）
./.claude/CLAUDE.md	← .claude内（Git共有）
./CLAUDE.local.md	← 個人用上書き（gitignore）
./src/CLAUDE.md	← サブディレクトリ（そのDir以下で有効）

重要: これらは全て **結合（concatenate）** されます。上位が下位を上書きするのではなく、全てが読み込まれます。

`AGENTS.md` という名前のファイルを置いている場合、`CLAUDE.md` と同様に扱われます（他のAIエージェントとの互換用）。両方ある場合は両方読まれます。

5-3. 作成タイミング

要件定義完了後、最初のコードを書く直前に作成します。要件が固まる前に書くと「とりあえず書いただけのルール」になりがち。設計を踏まえた具体的なルールを書きましょう。

5-4. CLAUDE.md 作成のプロンプト

Claude Codeへのプロンプト：

プロジェクトルートにCLAUDE.mdを作成してください。
先ほど合意した要件定義・設計ドキュメントの内容を反映し、
以下の全セクションを含めてください。

プロジェクト概要

- アプリ名、目的、ターゲットユーザーの1-2行の要約

技術スタック

- 使用する全技術とバージョン (Next.js 15, TypeScript 5.x, etc.)

ディレクトリ構成

```
src/
├── app/                                # App Router ページ
│   ├── (auth)/                        # 認証グループ (layout共有)
│   │   ├── login/
│   │   └── register/
│   ├── (main)/                        # メインアプリグループ
│   │   ├── dashboard/
│   │   ├── projects/
│   │   └── settings/
│   └── api/                            # Route Handlers
├── components/
│   ├── ui/                            # shadcn/ui コンポーネント
│   ├── features/                      # 機能別コンポーネント
│   │   ├── auth/
│   │   ├── project/
│   │   ├── task/
│   │   └── dashboard/
│   └── layouts/                        # レイアウトコンポーネント
├── lib/
│   ├── supabase/                      # Supabaseクライアント
│   ├── validations/                  # Zodスキーマ
│   └── utils/                         # ユーティリティ関数
├── hooks/                             # カスタムHooks
├── stores/                            # Zustandストア
└── types/                             # 型定義
```

コーディング規約

コンポーネント

- 関数コンポーネントのみ使用 (classコンポーネント禁止)
- named export を基本 (default export は page.tsx/layout.tsx のみ)
- Server Component をデフォルト、'use client' は必要な場合のみ
- Props型は ComponentName + Props で命名 (例: TaskCardProps)
- 1ファイル1コンポーネント
- コンポーネントファイルは PascalCase (例: TaskCard.tsx)

型定義

- interface を優先 (type は共用体型の場合のみ)
- any 型は絶対に使わない (unknown を使う)
- 型アサーション (as) は最小限に

インポート

- パスエイリアス @/ を必ず使用 (相対パス禁止)
- インポート順: React → 外部ライブラリ → 内部モジュール → 型
- 未使用インポートは残さない

スタイリング

- Tailwind CSS ユーティリティクラスのみ (カスタムCSS禁止)
- インラインスタイル禁止
- cn() ヘルパーでクラスを結合

状態管理

- サーバーの状態: Server Component + Supabase直接クエリ
- クライアントの状態: Zustand
- フォームの状態: react-hook-form
- URL状態: searchParams

エラーハンドリング

- error.tsx で画面単位のエラーをキャッチ
- not-found.tsx で404をハンドル
- loading.tsx でローディング状態を表示
- API Route ではtry-catch + 適切なHTTPステータスコードを返す
- ユーザー向けエラーはトースト (sonner) で表示

命名規約

種別	規則	例
コンポーネント	PascalCase	TaskCard.tsx
ユーティリティ	camelCase	formatDate.ts
定数	UPPER_SNAKE_CASE	MAX_FILE_SIZE
型/Interface	PascalCase	TaskStatus
Hooks	camelCase + use prefix	useTaskList.ts
Store	camelCase + Store suffix	taskStore.ts
API Route	route.ts (Next.js規約)	app/api/tasks/route.ts

Git ルール

- コミットメッセージは日本語で記述
- プレフィックス: feat:, fix:, refactor:, docs:, style:, test:, chore:
- 例: 「feat: タスク作成機能を追加」
- 1機能1コミットを原則
- .env ファイルは絶対にコミットしない

開発コマンド

- 開発サーバー: npm run dev
- ビルド: npm run build
- Lint: npm run lint
- 型チェック: npx tsc --noEmit

```

- テスト: npm test
- E2Eテスト: npm run test:e2e

# やってはいけないこと（絶対厳守）
- rm -rf や destructive なコマンドを実行しない
- .env ファイルを編集・作成しない
- any 型を使わない
- console.log をプロダクションコードに残さない
- インラインスタイルを使わない
- node_modules/ を Git にコミットしない
- 1ファイルが300行を超えないようにする（超える場合は分割）
- テストなしでコミットしない

# 設計ドキュメント参照
@docs/requirements.md
@docs/screens.md
@docs/database.md
@docs/api.md

```

最後の `@docs/requirements.md` のような行は、CLAUDE.md 内の **ファイル参照記法** です。`@` で始めるとそのファイルの中身がコンテキストに自動で取り込まれ、Claude がドキュメントを踏まえて作業できるようになります。

5-5. CLAUDE.local.md（個人設定）

チームメンバーそれぞれのスタイルに合わせた個人設定をCLAUDE.local.mdに配置できます（gitignoreされるため共有されない）。

Claude Codeへのプロンプト：

```

CLAUDE.local.md を作成してください。以下を記載：

- 回答は全て日本語で行うこと
- コードの説明は詳細に行うこと（初学者向け）
- 変更を加える前に必ず変更内容を説明し承認を得ること
- エラーが発生したら原因の推測と複数の解決案を提示すること

```

5-6. サブディレクトリ CLAUDE.md

特定のディレクトリ以下でのみ有効なルールを設定できます。サブディレクトリの CLAUDE.md は「そのフォルダ配下のファイルを触るとき」だけ自動で読み込まれます。

`src/app/api/CLAUDE.md`：

API Route 固有ルール

- 全てのRoute Handlerでzodバリデーションを実施
- 認証チェックを最初に行う
- レスポンス形式を統一: { data: T } | { error: string }
- HTTPステータスコードを正しく使う
 - 200: 成功
 - 201: 作成成功
 - 400: バリデーションエラー
 - 401: 認証エラー
 - 403: 認可エラー
 - 404: リソース未存在
 - 500: サーバーエラー

5-7. @ によるファイル参照

CLAUDE.md内で他のドキュメントを参照できます。

```
# 設計ドキュメント
@docs/requirements.md
@docs/database.md
@README.md
```

- **@<相対パス>**: そのファイルの中身を CLAUDE.md と一緒に読み込ませる。

注意: 参照先のファイルもコンテキストに読み込まれるため、大きすぎるファイルを参照するとコンテキストを圧迫します。1ファイル数千行になるようなものは、要点だけ抜粋した別ドキュメントを作って参照しましょう。

5-8. CLAUDE.md の更新タイミングと方法

タイミング	更新内容	プロンプト例
新パッケージ追加時	技術スタック更新	「CLAUDE.mdの技術スタックに lucide-react を追加して」
ディレクトリ構成変更時	構成図更新	「CLAUDE.mdのディレクトリ構成を現状に合わせて更新して」
新しい規約決定時	規約追加	「CLAUDE.mdに'Server Actionsは使わない'を追加して」
バグの再発防止時	禁止事項追加	「CLAUDE.mdに'useEffectでデータフェッチしない'を追加して」
大きなリファクタ後	全体見直し	「CLAUDE.mdを現在のプロジェクト状態に合わせて全体更新して」

特定項目の更新プロンプト:

CLAUDE.mdを更新してください。以下の変更を反映：

1. 技術スタックに追加：
 - @tanstack/react-query v5（サーバー状態管理）
 - sonner（トースト通知）
2. コーディング規約に追加：
 - データフェッチは React Query の useQuery を使用
 - Server Component での直接クエリは初回ロードのみ
 - useEffect でのデータフェッチは禁止
3. ディレクトリ構成に追加：
 - src/lib/queries/ - React Queryのクエリ関数

5-9. 特定のCLAUDE.mdを除外する

大きなモノレポで不要なCLAUDE.mdを除外：

```
{
  "claudeMdExcludes": [
    "**/node_modules/**/*.md",
    "**/other-project/CLAUDE.md"
  ]
}
```

- `claudeMdExcludes` : 読み込み対象から除外する glob パターンの配列。
- `**/node_modules/**` : あらゆる階層の node_modules を意味するパターン。

6. Phase 4：Rules の構成

6-1. Rules とは

`.claude/rules/` ディレクトリに配置するMarkdownファイルで、特定のファイルパスにマッチした時に自動的に適用されるルールです。CLAUDE.mdとは異なり、パスベースで条件分岐できます。

CLAUDE.md が「プロジェクト全体のルール」だとすれば、Rules は「このフォルダだけのルール」。たとえばコンポーネントを作るときと API Route を作る時では気をつけるポイントが違うので、別々の Rules ファイルに分けます。

6-2. Rules の仕組み

```
---
paths:
  # globパターンでマッチ
  - "src/components/**/*.tsx"
---

# ここにルールを記述
コンポーネント作成時のルール...
```

- `---` で囲まれた部分は YAML フロントマター（メタ情報）。
- `paths`: このルールを発火させるファイルパターン。glob 記法。
- `**` はあらゆる深さのフォルダにマッチ、`*` は任意のファイル名にマッチ。

Claudeが該当パスのファイルを操作する時、自動的にこのルールがコンテキストに追加されます。

6-3. Rules 作成のプロンプト

Claude Codeへのプロンプト：

.claude/rules/ ディレクトリに以下の4つのルールファイルを作成してください。
各ファイルにはpathsフロントマターとルール本文を含め、
具体的なコード例も必ず記載してください。

1. components.md

対象パス: src/components/**/*.tsx

含める内容:

- コンポーネントの雛形 (Server Component / Client Component)
- Props型の定義方法
- children の受け取り方
- 条件付きレンダリングのパターン
- forwardRef の使い方
- コンポーネント分割の基準 (いつ分割するか)
- shadcn/ui コンポーネントのカスタマイズ方法
- 良い例と悪い例の比較コード

2. api-routes.md

対象パス: src/app/api/**/*.ts

含める内容:

- Route Handler の雛形 (GET/POST/PATCH/DELETE)
- zodバリデーションの書き方
- 認証チェックのパターン
- エラーレスポンスの統一フォーマット
- Supabaseクエリの書き方
- ページネーションの実装パターン
- 良い例と悪い例の比較コード

3. database.md

対象パス: supabase/**/*.sql, src/lib/supabase/**/*.ts

含める内容:

- マイグレーションSQLの書き方
- RLSポリシーのテンプレート
- Supabaseクエリのベストプラクティス
- 型安全なクエリの書き方 (generated types使用)
- N+1クエリの回避方法

4. testing.md

対象パス: **/*.test.ts, **/*.test.tsx, **/*.spec.ts

含める内容:

- Vitestのテスト構造 (describe/it/expect)
- React Testing Libraryでのコンポーネントテスト
- モックの使い方 (vi.mock, vi.fn)
- API Routeのテスト方法

- テストデータのファクトリパターン
- テストの命名規則（日本語OK）
- AAA（Arrange-Act-Assert）パターン

6-4. Rules ファイルの具体例

`.claude/rules/components.md` :

```
---
paths:
  - "src/components/**/*.tsx"
---

# コンポーネント開発ルール

## Server Component（デフォルト）

```tsx
// Good: Server Component
// プロジェクト内の関数を @/ エイリアスで読み込む（相対パス禁止のため）
import { getProjects } from '@lib/queries/projects'

// Props 型は interface で定義。命名は ComponentName + Props
interface ProjectListProps {
 // 必須の文字列プロパティ
 userId: string
}

// named export。default export は page.tsx/layout.tsx のみ許可
// async が付くと Server Component で await が使える
export function ProjectList({ userId }: ProjectListProps) {
 // サーバー側でデータ取得。クライアントには結果だけ届く
 const projects = await getProjects(userId)

 return (

 { /* map で配列を要素に変換。key は React の差分検出に必須 */ }
 {projects.map((project) => (
 <li key={project.id}>{project.name}
))}

)
}
```

## Client Component（インタラクティブな場合のみ）

```
// Good: Client Component ('use client' を明示)
// ファイル先頭で書くと、このファイル全体がクライアント側でハイドレートされる
'use client'

// useState はクライアントでしか使えないので 'use client' が必要
import { useState } from 'react'
// shadcn/ui の Button コンポーネント
import { Button } from '@components/ui/button'

interface CounterProps {
 // 初期カウント値
 initialCount: number
}

export function Counter({ initialCount }: CounterProps) {
 // useState: 値とその更新関数のペアを返す
 const [count, setCount] = useState(initialCount)

 return (
 <div>
 <p>Count: {count}</p>
 {/* onClick は React のイベントハンドラ。アロー関数で書く */}
 <Button onClick={() => setCount((c) => c + 1)}>
 Increment
 </Button>
 </div>
)
}
```



## 禁止パターン

---

```
// Bad: any型 (型安全が崩れる)
function Component(props: any) { ... }

// Bad: default export (page.tsx以外)
export default function Component() { ... }

// Bad: useEffectでデータフェッチ (Server Component で十分)
useEffect(() => {
 fetch('/api/data').then(...)
}, [])

// Bad: インラインスタイル (Tailwind を使うべき)
<div style={{ color: 'red' }}>...</div>
```

## コンポーネント分割の基準

---

- 100行を超えたら分割を検討
- 200行を超えたら必ず分割
- 再利用される部分は即座に分割
- ロジック (hooks) とUI (component) を分離

```

`.claude/rules/api-routes.md`:

```markdown
---
paths:
  - "src/app/api/**/*.ts"
---

# API Route 開発ルール

## Route Handler の雛形

```ts
// Next.js のサーバー側型定義をインポート
import { NextRequest, NextResponse } from 'next/server'
// バリデーションライブラリ。スキーマで型と検証を同時に定義
import { z } from 'zod'
// Supabase の Server クライアント (cookie ベースの認証情報を持つ)
import { createClient } from '@lib/supabase/server'

// 1. バリデーションスキーマ定義
// z.object() で各フィールドの型・制約を宣言的に書く
const createTaskSchema = z.object({
 // 1~100文字の文字列
 title: z.string().min(1).max(100),
 // 任意、最大1000文字
 description: z.string().max(1000).optional(),
 // 列挙型
 status: z.enum(['todo', 'in_progress', 'review', 'done']).default('todo'),
 priority: z.enum(['low', 'medium', 'high']).default('medium'),
 // UUID 形式の文字列
 assigneeId: z.string().uuid().optional(),
 // ISO8601 形式
 dueDate: z.string().datetime().optional(),
})

// 2. Route Handler
// export async function POST で、このパスへの POST リクエストを処理
export async function POST(request: NextRequest) {
 try {
 // 3. 認証チェック
 // Supabase クライアント作成
 const supabase = await createClient()
 // 現在ログイン中のユーザーを取得
 const { data: { user }, error: authError } = await supabase.auth.getUser()

 // 未ログインなら 401 を返して終了
 if (authError || !user) {
 return NextResponse.json(

```

```

 // ユーザー向けメッセージ
 { error: '認証が必要です' },
 // HTTP ステータス
 { status: 401 }
)
}

// 4. リクエストバリデーション
// JSON ボディをオブジェクトに
const body = await request.json()
// safeParse は throw せず { success, data | error } を返す
const parsed = createTaskSchema.safeParse(body)

if (!parsed.success) {
 return NextResponse.json(
 { error: 'バリデーションエラー', details: parsed.error.flatten() },
 // クライアント側の入力ミスは 400
 { status: 400 }
)
}

// 5. DB操作
// from('tables').insert(...).select().single() の流れ
const { data, error } = await supabase
 // 対象テーブル
 .from('tasks')
 // 検証済みデータ + 作成者
 .insert({ ...parsed.data, created_by: user.id })
 // 挿入結果を取得
 .select()
 // 1件だけ取り出す（複数返ったらエラー）
 .single()

if (error) {
 return NextResponse.json(
 { error: 'タスクの作成に失敗しました' },
 // サーバー側エラーは 500
 { status: 500 }
)
}

// 6. 成功レスポンス
// 新規作成成功は 201 Created
return NextResponse.json({ data }, { status: 201 })
} catch {
 // 想定外の例外は 500 にまとめる
 return NextResponse.json(
 { error: '予期しないエラーが発生しました' },
 { status: 500 }
)
}

```

```
}
}
}
```

## レスポンス形式（統一）

---

- 成功: `{ data: T }`
- エラー: `{ error: string, details?: unknown }`
- 一覧: `{ data: T[], count: number, page: number, limit: number }`

---

### ## 7. Phase 5: Skills の作成と活用

#### ### 7-1. Skills とは

Skillsは\*\*再利用可能なプロンプトテンプレート\*\*です。`/skill-name` のスラッシュコマンドとして呼び出せます。繰り返し行う定型作業を自動化できます。

「同じような指示文を毎回書くのは面倒」「チームで指示の表現を揃えたい」というときに、Skill としてまとめておけば `/review` のような短いコマンドで呼び出せます。

#### ### 7-2. Skill の配置場所

~/claude/skills//SKILL.md ← 全プロジェクト共通 .claude/skills//SKILL.md ← プロジェクト固有

Skill 1つにつき1フォルダ作り、その中に `SKILL.md` を置きます。同じフォルダにヘルパースクリプトや参考資料を入れることもできます。

### ### 7-3. SKILL.md のフロントマター

```
```yaml
---
# 一意な識別子（小文字、ハイフン、最大64文字）
name: skill-id
# 呼び出し条件の判断に使われる（250文字以内）
description: "説明文"
# 引数のヒント（オートコンプリート用）
argument-hint: "[filepath]"
# true=手動呼び出しのみ、false=Claudeが自動呼び出し可
disable-model-invocation: false
# true=ユーザーが/で呼び出せる
user-invocable: true
# 使用可能なツール（スペース区切り）
allowed-tools: "Read Grep Edit"
# 使用モデル
model: claude-opus-4-6
# 推論レベル（low/medium/high/max）
effort: high
# forkでサブエージェントとして実行
context: fork
# サブエージェントタイプ
agent: Explore
# 自動発火のglobパターン
paths: ["src/**/*.tsx"]
---
```

各キーの意味は以下のとおりです。

- **name** : スラッシュコマンド名のもと（例 **review** → **/review**）。
- **description** : Claude が「いまこのスキル使うべき？」と判断する材料。
- **argument-hint** : 補完時に表示する引数の説明（例 **[filepath]**）。
- **disable-model-invocation** : Claude が自動でこの Skill を呼ぶのを禁止するか。
- **user-invocable** : ユーザーがスラッシュコマンドとして呼べるか。
- **allowed-tools** : この Skill 内で使えるツールを限定。
- **model** : 使うモデル（Opus / Sonnet など）。
- **effort** : 推論の深さ。high のほうが時間とコストがかかるが品質が上がる。
- **context** : **fork** でサブエージェントとして独立コンテキストで実行。メインの会話を汚さない。
- **agent** : ビルトインのサブエージェントタイプ。
- **paths** : 該当パターンのファイルを編集するとき自動的に呼ばれる。

7-4. 変数の使い方

変数	説明
<code>\$ARGUMENTS</code>	ユーザーが渡した引数全体
<code>\$0</code> , <code>\$1</code> , <code>\$2</code>	位置引数
<code>\${CLAUDE_SESSION_ID}</code>	セッションID
<code>\${CLAUDE_PROJECT_DIR}</code>	プロジェクトルート
<code>\${CLAUDE_SKILL_DIR}</code>	Skillファイルのディレクトリ
<code>!`command`</code>	コマンド実行結果を注入（Skill読み込み時に実行）

`!`コマンド`` は「Skill が読み込まれた瞬間にそのコマンドを実行し、結果をその場に埋め込む」記法です。例: `!`git diff --name-only`` と書けば、Skill 起動時の変更ファイル一覧がそのままプロンプトに入ります。

7-5. Skills 作成のプロンプト

Claude Codeへのプロンプト：

以下の4つのSkillを `.claude/skills/` に作成してください。
各SkillにはSKILL.mdを作成し、フロントマターと
詳細なプロンプトテンプレートを含めてください。

Skill 1: /review (コードレビュー)

- 名前: `review`
- 引数: ファイルパス (省略可、省略時は`git diff`の変更ファイル全て)
- 機能: コードをレビューし、セキュリティ・パフォーマンス・規約の3観点で問題を検出・修正提案
- ツール: `Read`, `Grep`, `Glob`
- モデル: `claude-opus-4-6`
- context: `fork` (メインコンテキストを汚さない)

Skill 2: /create-component (コンポーネント生成)

- 名前: `create-component`
- 引数: コンポーネント名
- 機能: CLAUDE.mdとRulesに準拠したコンポーネントの雛形を生成
- ツール: `Read`, `Write`, `Edit`
- 生成ファイル:
 - `src/components/features/[name]/[Name].tsx`
 - `src/components/features/[name]/[Name].test.tsx`

Skill 3: /create-api (API Route生成)

- 名前: `create-api`
- 引数: リソース名 (例: `tasks`)
- 機能: CRUD全操作のRoute Handlerを雛形生成
- ツール: `Read`, `Write`, `Edit`
- 生成ファイル:
 - `src/app/api/[name]/route.ts` (`GET`, `POST`)
 - `src/app/api/[name]/[id]/route.ts` (`GET`, `PATCH`, `DELETE`)
 - `src/lib/validations/[name].ts` (`zod`スキーマ)

Skill 4: /simplify (コード改善)

- 名前: `simplify`
- 引数: フォーカス領域 (省略可)
- 機能: 変更されたコードの重複排除・リファクタリング・効率改善
- ツール: `Read`, `Grep`, `Edit`
- context: `fork`

7-6. Skill の具体例

`.claude/skills/review/SKILL.md` :

```

---
name: review
description: コードレビューを実施。セキュリティ・パフォーマンス・規約の3観点で問題を検出し修正提案を行う。
argument-hint: "[filepath or empty for git diff]"
user-invocable: true
allowed-tools: Read Grep Glob Bash
model: claude-opus-4-6
effort: high
context: fork
---

# コードレビュータスク

## レビュー対象
$ARGUMENTS が指定されている場合はそのファイルをレビュー。
指定がない場合は `git diff --name-only` の出力ファイルを全てレビュー。

## レビュー観点（全て確認すること）

### 1. セキュリティ (Critical)
- [ ] SQLインジェクションの可能性
- [ ] XSS (Cross-Site Scripting) の可能性
- [ ] 認証チェックの漏れ（全API Routeで実施されているか）
- [ ] 認可チェックの漏れ（RLSが適切か）
- [ ] 機密情報のハードコード（APIキー、パスワード等）
- [ ] CSRF対策

### 2. パフォーマンス (High)
- [ ] 不要な再レンダリング（memo, useMemo, useCallback の適切な使用）
- [ ] N+1クエリの有無
- [ ] 大きなバンドルサイズ（dynamic importの検討）
- [ ] 画像の最適化（next/image使用）
- [ ] 不要な'use client'（Server Componentで可能なものはないか）

### 3. コーディング規約 (Medium)
- [ ] CLAUDE.mdの規約に準拠しているか
- [ ] 命名規約に従っているか
- [ ] any型が使われていないか
- [ ] console.logが残っていないか
- [ ] 未使用のインポートがないか
- [ ] ファイルサイズが300行以内か

### 4. エラーハンドリング (Medium)
- [ ] try-catchが適切に使われているか
- [ ] ユーザー向けエラーメッセージが適切か
- [ ] ローディング状態が処理されているか
- [ ] 空状態が処理されているか

```


5. アクセシビリティ (Low)

- [] 適切なaria属性
- [] キーボード操作可能
- [] 画像のalt属性
- [] フォームのlabel

出力形式

各問題を以下の形式で報告：

****[重要度: Critical/High/Medium/Low]**** ファイル名:行番号

- 問題：何が問題か
- 理由：なぜ問題か
- 修正案：具体的なコード

.claude/skills/create-component/SKILL.md :

```

---
name: create-component
description: CLAUDE.mdとRulesに準拠したReactコンポーネントの雛形を生成する。ファイルとテストを同時に作成。
argument-hint: <ComponentName>
user-invocable: true
allowed-tools: Read Write Edit Bash
model: claude-sonnet-4-6
effort: medium
---

# コンポーネント生成タスク

## 指示
「$ARGUMENTS」という名前のコンポーネントを生成してください。

## 生成前の確認
1. CLAUDE.md を読んでコーディング規約を確認
2. .claude/rules/components.md を読んでコンポーネントルールを確認
3. 既存の類似コンポーネントがあれば参考にする

## 生成ファイル

### 1. コンポーネント本体
パス: `src/components/features/$ARGUMENTS/$ARGUMENTS.tsx`

- Props interfaceを定義（${ARGUMENTS}Props）
- Server Component をデフォルト（インタラクティブなら'use client'追加）
- named export
- Tailwind CSSでスタイリング
- 適切なaria属性を含める

### 2. テストファイル
パス: `src/components/features/$ARGUMENTS/$ARGUMENTS.test.tsx`

- 基本的なレンダリングテスト
- Props変更時のテスト
- ユーザーインタラクションテスト（該当する場合）

## 生成後
- 生成したファイルの内容を簡潔に説明
- 使い方の例を1つ提示

```

7-7. ビルトインSkills

Claude Codeには組み込みのSkillもあります。

Skill	用途	使い方
<code>/commit</code>	変更をコミット	<code>/commit</code>
<code>/simplify</code>	コード改善レビュー	<code>/simplify focus on performance</code>
<code>/loop</code>	定期実行	<code>/loop 5m check if deploy finished</code>
<code>/claude-api</code>	Claude APIリファレンス読み込み	<code>/claude-api</code>
<code>/frontend-design</code>	高品質UIコンポーネント生成	自動発火（UI実装時）

8. Phase 6：Subagents（カスタムエージェント）の構成

8-1. Subagents とは

Subagentsは**独立したAIエージェント**を定義し、特定のタスクを委任できる仕組みです。メインのコンテキストを汚さずに、専門的なタスクを並列実行できます。

たとえば「コードレビュー専門のエージェント」「テスト作成専門のエージェント」をそれぞれ用意し、必要に応じて使い分けると、メイン会話のコンテキストを節約しつつ専門タスクを任せられます。

8-2. 配置場所

```
~/.claude/agents/<name>.md    ← 全プロジェクト共通
.claude/agents/<name>.md      ← プロジェクト固有
```

8-3. Subagent のフロントマター

```
---
# 一意な識別子
name: agent-id
# いつ委任するか判断基準
description: "説明文"
# 使用モデル
model: claude-opus-4-6
# 許可するツール
tools: Read Bash Edit Grep
permissions:
  allow: ["Bash(npm test*)"]
  deny: ["Edit(/.env)", "WebFetch"]
# プリロードするSkill
skills: [review, create-component]
memory:
  # メモリ使用の可否
  enabled: true
# worktreeで並列実行
isolation: worktree
---
```

- **name** : エージェント識別子。
- **description** : メインのClaude がこのエージェントへ委任するか判断する材料。
- **model** : 使用するモデル。
- **tools** : このエージェント内で許可するツール（スペース区切り）。
- **permissions.allow** / **deny** : ツール実行の許可・拒否。
- **skills** : 起動時にあらかじめ読み込む Skill。
- **memory.enabled** : メモリの使用可否。
- **isolation: worktree** : worktree で別作業ツリーを使って並列実行。

8-4. ビルトインSubagent

Agent	用途	context指定
Explore	コードベース調査・検索	context: fork + agent: Explore
Plan	タスク計画の立案	Plan Mode使用時
general-purpose	汎用（デフォルト）	指定なし

8-5. Subagent 作成のプロンプト

Claude Codeへのプロンプト：

.claude/agents/ に以下の2つのカスタムSubagentを作成してください。

Agent 1: code-reviewer.md

- 名前: code-reviewer
- 目的: コードレビューの専門エージェント
- モデル: claude-opus-4-6
- 許可ツール: Read, Bash, Grep, Glob (Editは禁止)
- 禁止: WebFetch, Edit (読み取り専用)
- 動作:
 1. 指定されたファイルまたはgit diffを分析
 2. セキュリティ・パフォーマンス・規約の観点でレビュー
 3. 問題を重要度別に分類して報告
 4. 具体的な修正コードを提示 (ただし自分では編集しない)

Agent 2: test-writer.md

- 名前: test-writer
- 目的: テストコード作成の専門エージェント
- モデル: claude-sonnet-4-6 (コスト効率重視)
- 許可ツール: Read, Write, Edit, Bash, Grep
- 許可Bash: npm test*, npx vitest*
- 動作:
 1. 対象コンポーネント/関数のソースコードを読む
 2. 既存のテストパターンを調査
 3. Vitest + React Testing Library でテストを作成
 4. テストを実行して全パスすることを確認

8-6. Subagent の具体例

`.claude/agents/code-reviewer.md` :

```
---
name: code-reviewer
description: コードレビューの専門エージェント。セキュリティ、パフォーマンス、規約準拠を検査し、問題を報告する。自身ではコードを編集しない。
model: claude-opus-4-6
tools: Read Bash Grep Glob
permissions:
  deny: ["Edit", "Write", "WebFetch"]
memory:
  enabled: false
---
```

あなたはコードレビューの専門家です。

あなたの役割

- コードを分析して問題を発見する
- 具体的な修正案を提示する
- ただし、自分ではコードを編集しない（読み取り専用）

レビュー手順

1. まず `CLAUDE.md` を読んでプロジェクトの規約を理解する
2. `.claude/rules/` のルールファイルを確認する
3. 対象コードを精読する
4. 以下の観点でレビューする

レビュー観点

Critical（即座に修正必要）

- セキュリティ脆弱性（XSS, SQLi, 認証漏れ）
- データ損失の可能性
- 本番障害に直結する問題

High（早急に修正推奨）

- パフォーマンス問題
- エラーハンドリングの欠如
- 型安全性の問題

Medium（改善推奨）

- コーディング規約違反
- 可読性の問題
- テストの不足

Low（余裕があれば改善）

- アクセシビリティ
- 命名の改善提案
- コメントの追加

出力形式

問題ごとに以下の形式で報告すること：

[Critical/High/Medium/Low] 問題のタイトル

- ファイル: path/to/file.ts:行番号
- 問題: 具体的に何が問題か
- 理由: なぜこれが問題か
- 修正案: `typescript` // 修正後のコード

9. Phase 7：開発フェーズ — 機能実装の進め方

9-1. 実装順序（推奨）

Phase 7-1: 基盤構築

- └─ 共通レイアウト
- └─ DB接続
- └─ 共通UIコンポーネント
- └─ 認証基盤

↓

Phase 7-2: 認証機能

- └─ ログイン
- └─ 新規登録
- └─ ログアウト
- └─ セッション管理

↓

Phase 7-3: 主要CRUD（最もシンプルなりソース）

- └─ プロジェクトCRUD
- └─ タスクCRUD

↓

Phase 7-4: 高度な機能

- └─ カンバンボード（D&D）
- └─ フィルタリング・検索
- └─ ダッシュボード

↓

Phase 7-5: UI/UX改善

- └─ アニメーション
- └─ ローディング/エラー/空状態
- └─ レスポンシブ対応
- └─ ダークモード

↓

Phase 7-6: テスト

- └─ ユニットテスト
- └─ コンポーネントテスト
- └─ E2Eテスト

↓

Phase 7-7: デプロイ・最終調整

「土台 → 認証 → CRUD → 派生機能 → UI磨き → テスト → デプロイ」という順序が王道です。順序を飛ばすと後で痛い目を見ます（例: 認証より先にCRUDを書くと、後でRLSの入れ込みに苦勞する）。

9-2. 1機能の実装フロー（黄金パターン）

Step 1: 仕様の説明（プロンプト）

「〇〇機能を実装します。仕様は以下...」
「まだコードは書かないで、実装方針を説明して」

Step 2: Claude が実装方針を回答

「以下の手順で実装します: 1. ... 2. ... 3. ...」

Step 3: 方針を確認・修正指示

「方針 OK です。ただし〇〇は△△に変更して」
または「いいですね、進めてください」

Step 4: 実装（Claude がコード生成）

※ 大きい機能は分割して段階的に実装
「まず〇〇だけ実装して。完了したら止まって」

Step 5: 動作確認

`npm run dev` でブラウザ確認
エラーがあれば詳細を伝えて修正依頼

Step 6: コードレビュー

`/review` を実行
または手動でレビュー依頼

Step 7: テスト作成

`/create-component` で雛形があればテストも含まれる
追加テストが必要なら test-writer agent に委任

Step 8: コミット

`/commit` を実行
日本語メッセージ + プレフィックス
※ コミット前に必ず `git diff` を自分の目で確認

[次の機能へ](#)

`/commit` を AI に任せきりにせず、必ず `git diff` で内容を確認してから走らせるのがおすすめです。push は人間が手動で行うのが基本。

9-3. 精度を最大化するプロンプトテンプレート

```
## 実装する機能
[機能名を1行で明確に]

## この機能の目的
[なぜこの機能が必要か。ユーザーストーリー形式で]
「〇〇として、△△したい。なぜなら□□だから。」

## 具体的な仕様
### 画面
- どの画面で使うか（URL含む）
- 画面のどの位置に配置するか

### データ
- 使用するテーブル/APIエンドポイント
- 必要なクエリ（SELECT/INSERT/UPDATE/DELETE）

### ユーザー操作フロー
1. ユーザーが〇〇をクリック
2. △△が表示される
3. ユーザーが□□を入力
4. ××ボタンを押す
5. 成功時：〇〇が表示される
6. 失敗時：エラーメッセージが表示される

### 状態管理
- ローディング中：スケルトンUI
- データなし：「まだ〇〇がありません」+ 作成ボタン
- エラー時：エラーメッセージ + リトライボタン
- 成功時：トースト通知「〇〇しました」

## エッジケース
- [入力が空の場合]
- [文字数上限の場合]
- [同時編集が発生した場合]
- [ネットワーク切断時]
- [認証切れの場合]

## 制約
- Server Component / Client Component どちらを使うか
- 使用するUI部品（shadcn/uiのどのコンポーネント）
- パフォーマンス要件（遅延読み込みの可否）

## 参考
- 類似の既存実装：[ファイルパス]
- docs/api.md の該当セクション

## 進め方
```

まずコードは書かずの実装方針を説明してください。
合意後に実装に移ります。

このテンプレを埋めていくだけで、AIの出力品質が劇的に上がります。

9-4. 実装例：基盤構築

Claude Codeへのプロンプト：

Phase 7-1: 基盤構築を開始します。以下を順番に実装してください。

1. 共通レイアウト（まずこれだけ）

src/app/layout.tsx を以下の仕様で実装：

- `html lang="ja"`
- フォント: Inter (`next/font/google`)
- `ThemeProvider` でダークモード対応 (`next-themes`)
- `Toaster (sonner)` をグローバルに配置

src/components/layouts/AppLayout.tsx:

- ヘッダー: ロゴ、ナビゲーション、ユーザーメニュー
- サイドバー: デスクトップ時に表示、モバイルではハンバーガーメニュー
- メインコンテンツエリア
- レスポンシブ対応

shadcn/ui から必要なコンポーネントをインストールしてください。

完了したら一度止まってください。ブラウザで確認します。

「完了したら一度止まってください」が重要。これにより段階的にレビューでき、問題の早期発見が可能です。

9-5. 効果的な修正指示の出し方

[悪い例]

「動かないので直してください」
「エラーが出ます」
「なんか変です」

[良い例]

「TaskCardコンポーネントで以下のエラーが出ています。

エラーメッセージ:

`TypeError: Cannot read properties of undefined (reading 'name')`

発生場所:

`src/components/features/task/TaskCard.tsx:25`行目

再現手順:

1. `/projects/1/board` を開く
2. 担当者未設定のタスクが表示されるとクラッシュ

期待する動作:

担当者が未設定の場合は「未割り当て」と表示される

原因の推測:

`task.assignee` が `null` の場合の条件分岐が漏れている

このエラーを修正してください。」

エラーメッセージ全文・ファイル名・行番号・再現手順・期待動作の5点セットを揃えると、AI のデバッグ精度が一段上がります。

9-6. 大きすぎる変更を制御する方法

ストップ。変更範囲が大きすぎます。

以下のように分割して、1つずつ実装してください：

1. まずTaskCardコンポーネントのUI部分だけ
2. 確認後、TaskFormコンポーネント
3. 確認後、カスタムHook (`useTaskList`)
4. 確認後、API Route
5. 確認後、全体の結合

今は1だけ実装してください。2以降は指示があるまで待ってください。

「ストップ」と明確に伝えると、Claude は途中でも作業を中断して指示を仰ぎます。

10. Phase 8：テスト・品質管理

10-1. テスト環境のセットアップ

Claude Codeへのプロンプト：

テスト環境をセットアップしてください。

パッケージ

- vitest
- @testing-library/react
- @testing-library/jest-dom
- @testing-library/user-event
- @vitejs/plugin-react
- jsdom
- playwright (E2E用)

設定ファイル

1. vitest.config.ts
 - React プラグイン
 - jsdom 環境
 - パスエイリアス (@/) 対応
 - カバレッジ設定 (v8 provider)
2. src/test/setup.ts
 - @testing-library/jest-dom のインポート
 - グローバルモック (next/navigation等)
3. playwright.config.ts
 - ベースURL: http://localhost:3000
 - ブラウザ: chromium
 - スクリーンショット: on failure

npmスクリプト (package.jsonに追加)

- "test": "vitest run"
- "test:watch": "vitest"
- "test:coverage": "vitest run --coverage"
- "test:e2e": "playwright test"
- "test:e2e:ui": "playwright test --ui"

セットアップ後、サンプルテストを1つ作成して
npm test が通ることを確認してください。

10-2. テスト作成の依頼

Claude Codeへのプロンプト（または `/review` Skill を使用）：

タスクCRUD機能のテストを作成してください。

テスト対象と種別

ユニットテスト (src/lib/validations/task.test.ts)

- zodスキーマのバリデーション
 - 正常なデータ → パス
 - タイトル空 → エラー
 - タイトル101文字 → エラー
 - 不正なステータス値 → エラー
 - 不正なUUID → エラー

コンポーネントテスト (src/components/features/task/)

- TaskCard.test.tsx
 - 正常にレンダリングされる
 - タイトル、担当者、期限が表示される
 - 担当者未設定時は「未割り当て」が表示される
 - クリックで onSelect が呼ばれる
 - 優先度バッジの色が正しい
- TaskForm.test.tsx
 - フォームが正常に表示される
 - 空のタイトルで送信するとバリデーションエラー
 - 正常な値で送信すると onSubmit が呼ばれる
 - キャンセルボタンで onCancel が呼ばれる

API Routeテスト (src/app/api/tasks/route.test.ts)

- POST: 正常なリクエスト → 201
- POST: バリデーションエラー → 400
- POST: 認証なし → 401
- GET: タスク一覧取得 → 200
- PATCH: タスク更新 → 200
- DELETE: ソフトデリート → 200

テストはAAA (Arrange-Act-Assert) パターンで書いてください。

テスト名は日本語で記述してください。

作成後 npm test を実行して全テストがパスすることを確認してください。

AAA は「Arrange (準備) → Act (実行) → Assert (検証)」の3ステップでテストを書く方針です。1テスト1観点・前提と検証が一目で分かる構造を目指します。

11. Phase 9：デプロイと最終調整

11-1. デプロイ前チェック

Claude Codeへのプロンプト：

デプロイ前の最終チェックを実行してください。

1. ビルド確認
npm run build を実行してエラーがないか確認
2. 型チェック
npx tsc --noEmit を実行
3. Lint
npm run lint を実行
4. テスト
npm test を実行
5. コードスキャン
以下をgrepで検索して該当があれば修正：
 - console.log (テストファイル以外)
 - any型 (テストファイル以外)
 - TODO/FIXME コメント (一覧表示)
 - ハードコードされたURL/APIキー
6. 環境変数チェック
 - .env.example に全ての必要な変数が記載されているか
 - .env が .gitignore に含まれているか
7. セキュリティチェック
 - npm audit を実行
 - 脆弱性があれば対応

全ての結果をレポートしてください。問題があれば修正してください。

デプロイ前にこのチェックリストを実行する習慣を持つだけで、本番事故がぐっと減ります。

12. Context管理とMemoryシステム

12-1. Contextとは

Claude Codeのコンテキストウィンドウには上限があります。長い会話を続けると、古いメッセージが自動的に圧縮（compact）されます。

前述のとおりコンテキストウィンドウは AI が一度に扱えるトークン数の上限のことです。会話・読み込みファイル・指示すべてがこの「容器」に入り、満杯になると古いものから消えたり要約されたりします。

12-2. Context管理コマンド

コマンド	説明
<code>/compact</code>	手動でコンテキストを圧縮（会話を要約して容量削減）
<code>/clear</code>	コンテキストを完全にクリア（新しいセッション開始）
<code>/cost</code>	現在のトークン使用量とコストを確認

12-3. `/compact` の使いどころ

以下のタイミングで `/compact` を実行する：

- 1つの機能の実装が完了し、次の機能に移る時
- 長い議論が終わって結論が出た時
- エラーの修正が完了した時
- コンテキストウィンドウの使用率が高くなった時

注意: `/compact` を実行するとCLAUDE.mdは再読み込みされますが、会話の細かいニュアンスは失われます。

`PostCompact` Hookで重要な情報を再注入すると良いです。

12-4. Memory システム

Claude Codeは会話を跨いで記憶を保持するMemoryシステムを持っています。コンテキストとは別の場所に保管され、セッションをまたいで読み込まれます。

保存場所： `~/.claude/projects/<project>/memory/`

```
memory/
├─ MEMORY.md          ← インデックス（セッション開始時に読み込み）
├─ user_preferences.md ← ユーザーの好み
├─ project_decisions.md ← 技術的な意思決定
└─ feedback_testing.md ← フィードバック記録
```

Memoryに保存すべき情報： - ユーザーの役割・スキルレベル - プロジェクト固有の意思決定とその理由 - フィードバック（「こうしてほしい/しないでほしい」） - 外部リソースへの参照

Memoryに保存すべきでない情報： - コードパターン（コードを読めばわかる） - Git履歴（git logで確認できる） - CLAUDE.mdに書いてあること

Memoryの操作プロンプト：

```
# 記憶させる
「覚えておいて：このプロジェクトではServer Actionsは使わず、
Route Handlersで統一する決定をした。理由はテストの書きやすさ。」

# 思い出させる
「前のセッションで決めたAPI設計方針を思い出して」

# 忘れさせる
「zustandを使う決定は撤回したので、その記憶を削除して」
```

Memoryの有効/無効設定：

```
{
  "autoMemoryEnabled": true
}
```

- `autoMemoryEnabled` : Claude が自動的に Memory を読み書きするか。 `false` にすると明示指示時のみ動く。

13. キーボードショートカットのカスタマイズ

13-1. 設定ファイル

場所： `~/.claude/keybindings.json`

13-2. デフォルトの主要ショートカット

キー	動作	コンテキスト
Enter	プロンプト送信	Chat
Escape	キャンセル/中断	Chat
Shift+Tab	パーミッションモード切り替え	Chat
Ctrl+C	実行中断	Global
Ctrl+D	終了	Global
Ctrl+L	入力クリア	Chat
Ctrl+O	詳細表示（thinking表示）切り替え	Global
Ctrl+R	履歴検索	Global
Alt+T (Win/Linux)	Thinking Mode 切り替え	Chat
Ctrl+G	外部エディタで編集	Chat

13-3. カスタマイズ例

```
{
  "$schema": "https://www.schemastore.org/claude-code-keybindings.json",
  "bindings": [
    {
      "context": "Chat",
      "bindings": {
        "ctrl+k": "chat:clearInput",
        "ctrl+e": "chat:externalEditor",
        "ctrl+u": null
      }
    },
    {
      "context": "Global",
      "bindings": {
        "ctrl+t": "app:toggleTodos",
        "ctrl+o": "app:toggleTranscript"
      }
    }
  ]
}
```

- **\$schema**: エディタが入力補完するための JSON Schema 参照。
- **bindings**: コンテキストごとのバインディング配列。

- `context` : バインディングが効く場面（Chat / Global など）。
- `"ctrl+k": "chat:clearInput"` : Ctrl+K に「入力欄をクリア」を割り当て。
- `"ctrl+u": null` : 既存のバインディングを無効化。

13-4. 利用可能なコンテキスト

コンテキスト	説明
Global	どこでも有効
Chat	メイン入力エリア
Autocomplete	補完メニュー表示中
Confirmation	権限確認ダイアログ
Transcript	詳細表示モード
HistorySearch	Ctrl+Rの履歴検索中
DiffDialog	diff表示中

13-5. コード記法

```
ctrl+k      # Control + K
shift+tab   # Shift + Tab
ctrl+shift+c # 複数修飾キー
ctrl+x ctrl+e # コード入力 (Ctrl+X → Ctrl+E)
K           # 大文字 = Shift暗黙
escape, enter # 特殊キー名
```

「コード入力」は2段押し（プレフィックスキーを押してから別のキーを押す）の記法です。Emacs 風の操作を好む人向け。

14. Worktrees（並列開発）

14-1. Worktrees とは

Git worktreeを使って、メインの作業を中断せずに別ブランチで並列作業できます。1つのリポジトリから複数の作業ディレクトリを派生させ、それぞれで別ブランチを進められる仕組みです。

14-2. 使い方

```
# 名前を指定してworktreeを作成
# --worktree オプションで feature-auth という新しい worktree を作る
claude --worktree feature-auth

# 自動で名前を生成
# 名前を省略するとランダムな名前で worktree を作成
claude --worktree

# tmux連携 (macOS/Linux)
# -w は --worktree の短縮形。--tmux で新しい tmux ペインに展開
claude -w feature-auth --tmux
```

worktreeの保存場所： `.claude/worktrees/<name>/`

Gitブランチ： `worktree-<name>` が自動作成（`origin/HEAD` ベース）

14-3. .worktreeinclude ファイル

gitignoreされたファイルをworktreeにコピーするための設定：

プロジェクトルートに `.worktreeinclude` を作成：

```
.env
.env.local
.claude/settings.local.json
.claude/.mcp.local.json
```

- 各行に「gitignore されているがworktreeにはコピーしたい」ファイル名を書く。
- `.env` などはGit追跡から外れているが、worktreeで開発するときに必要なのでこの仕組みでコピーする。

gitignoreされた **かつ** .worktreeincludeにマッチするファイルのみコピーされます。

14-4. 活用シーン

シーン1: メイン機能の開発中にバグ修正が必要

→ `claude -w hotfix-auth` で別worktreeを作成し、バグ修正

シーン2: 2つの機能を同時に進めたい

→ Terminal 1: `claude -w feature-kanban`

→ Terminal 2: `claude -w feature-dashboard`

シーン3: 破壊的な変更を試したい

→ `claude -w experiment-new-ui` で実験

→ 問題なければマージ、ダメなら破棄

15. CLIフラグとオプション一覧

15-1. セッション管理

```
# 新しいセッション開始 (最も普通の起動方法)
claude
# 前回のセッションを再開 (-c は continue の意味)
claude -c
# 名前指定でセッション再開
claude -r "session-name"
# セッションに名前をつけて開始 (後で -r で呼び戻せる)
claude -n "my-feature"
# Pull Request #123 連携セッションを再開
claude --from-pr 123
```

15-2. モードとモデル

```
# Plan Mode で起動 (読み取り専用)
claude --permission-mode plan
# 編集は確認なしで自動承認 (信頼できるタスク向け)
claude --permission-mode acceptEdits
# Auto Mode (バックグラウンドで安全性チェック)
claude --permission-mode auto
# Opus モデルを使用 (高品質・高コスト)
claude --model opus
# Sonnet モデルを使用 (安価・高速)
claude --model sonnet
# 推論レベルを最高に上げる
claude --effort high
```

15-3. 非対話モード (スクリプト連携)

```
# -p は print mode。質問を投げて回答だけ受け取る (対話なし)
claude -p "このプロジェクトの構成を教えて"

# JSON 出力に切り替え。スクリプトでパースしやすい
claude -p "package.jsonの依存関係を一覧して" --output-format json

# 最大ターン数を制限 (無限ループ防止)
claude -p "テストを全部実行して" --max-turns 5

# 予算上限を USD で指定。超えそうになると停止
claude -p "全機能を実装して" --max-budget-usd 5.00
```

15-4. コンテキストとツール

```
# 別ディレクトリ（共有ライブラリ等）もコンテキストに含める
claude --add-dir ../shared-lib

# 使用ツール制限（カンマ区切り）
claude --tools "Bash,Edit,Read"
# 特定ツールだけ禁止
claude --disallowedTools "WebFetch"

# システムプロンプトを上書き（Claude の根本的な指示を変える）
claude --system-prompt "You are a TypeScript expert"
# システムプロンプトをファイルから読み込み
claude --system-prompt-file ./custom-prompt.txt

# MCP 設定ファイルを個別に指定
claude --mcp-config ./custom-mcp.json
```

15-5. デバッグ

```
# デバッグモード（内部の挙動を詳細出力）
claude --debug
# デバッグログをファイルに記録
claude --debug-file /tmp/claude.log
# 通常より詳細な出力
claude --verbose
```

16. トラブルシューティング

16-1. よくある問題と解決法

問題	原因	解決法
Claudeが同じ変更を繰り返す	コンテキスト不足	<code>/compact</code> して明確に再指示
生成コードの品質が低い	指示が曖昧	具体的な仕様とコード例を提示
意図と異なるファイルを編集	パスの認識ずれ	絶対パスを明示して指示
コンテキストが足りない	会話が長すぎる	<code>/compact</code> or <code>/clear</code> で新セッション
Hookが動かない	matcherの不一致	<code>--debug</code> でHookの発火を確認
MCPサーバーに接続できない	設定ミス	<code>claude mcp list</code> で状態確認
Permission denied	deny設定	<code>settings.json</code> のdenyリストを確認

16-2. コード品質が低い場合の改善指示

生成されたコードに以下の問題があります。修正してください：

- コンポーネントが大きすぎる（300行超）
 - 責務ごとに分割してください
 - UIとロジック（カスタムHook）を分離してください
- ロジックがコンポーネントに直書き
 - カスタムHookに抽出してください
 - ファイル： `src/hooks/use[Feature].ts`
- マジックナンバーがある
 - 定数ファイルに定義してください
 - ファイル： `src/lib/constants.ts`
- 型定義が甘い
 - `any/unknown` を具体的な型に変更
 - 型ファイル： `src/types/[feature].ts`

CLAUDE.mdの規約に準拠しているか再確認してから修正してください。

16-3. Claude が暴走した場合

ストップ。

今行った変更を全て説明してください。
どのファイルを、どのように変更しましたか？
一覧にして報告してください。

報告後、必要な修正があれば指示します。
勝手に追加の変更はしないでください。

それでも止まらない場合は、`Ctrl+C` で実行を強制中断し、`git status` と `git diff` で被害状況を確認してから、必要に応じて `git restore` や `git checkout` で巻き戻します（破壊的コマンドは事前にコミットしてあれば安全に戻せます）。

17. ベストプラクティス総まとめ

17-1. プロンプトの黄金ルール 10箇条

#	ルール	具体例
1	具体的に指示する	「いい感じに」→「タイトルは1-100文字、zodバリデーション付き」
2	段階的に進める	「全部作って」→「まずレイアウトだけ。確認後に次へ」
3	コンテキストを与える	「ボタンを作って」→「カンバンボードのタスク作成ボタン。クリックでモーダルが開く」
4	制約を明示する	「フォーム作って」→「react-hook-form + zod。Server Actionsは使わない」
5	レビューを挟む	各機能実装後に <code>/review</code> を実行
6	エッジケースを示す	「データなし・ローディング・エラー時の表示も実装して」
7	参照を示す	「src/components/features/project/ProjectCard.tsxを参考にして」
8	まず方針を確認	「まだコードは書かないで。実装方針を説明して」
9	停止ポイントを設ける	「完了したら一度止まって。ブラウザで確認します」
10	エラーは詳細に伝える	エラーメッセージ + ファイル + 行番号 + 再現手順

17-2. 機能別の推奨設定

機能	フェーズ	設定場所
危険コマンドのブロック	Phase 0（最初）	<code>~/.claude/settings.json</code>
MCP連携	Phase 0	<code>~/.claude/.mcp.json</code>
Hooks（自動フォーマット等）	Phase 0	<code>settings.json</code>
Keybindings	Phase 0	<code>~/.claude/keybindings.json</code>
CLAUDE.md	Phase 3（要件確定後）	プロジェクトルート
Rules	Phase 4（実装開始前）	<code>.claude/rules/</code>
Skills	Phase 5（開発中）	<code>.claude/skills/</code>
Subagents	Phase 6（開発中）	<code>.claude/agents/</code>
Memory	随時（自動）	<code>~/.claude/projects/*/memory/</code>
Worktrees	必要時（並列開発）	CLIフラグ

17-3. やりがちな失敗と対策

失敗パターン	対策
要件を曖昧にしたまま実装開始	Plan Modeで要件を先に固める
CLAUDE.mdを作らない/更新しない	Phase 3で作成し、変更の度に更新
一度に大量の機能を依頼	1機能ずつ実装→確認→コミット
エラーの詳細を伝えない	エラーメッセージ、ファイル、行番号を正確に
レビューを省略	<code>/review</code> Skillで毎回レビュー
Gitコミットを溜める	<code>/commit</code> で機能単位にこまめに
コンテキストを管理しない	機能完了ごとに <code>/compact</code>
Rulesを活用しない	パス別ルールで規約自動適用
Skillsを作らない	繰り返す操作はSkill化して効率化
全てデフォルトモードで実行	設計=Plan、実装=acceptEdits、レビュー=Plan
AIにそのまま git push させる	コミット内容を確認し、push は人間が手動で行う
<code>git status</code> を確認せず作業開始	毎セッション最初に <code>git status</code> / <code>git diff</code> で現状把握

17-4. 推奨ワークフロー全体図



Phase 8-9: テスト・デプロイ

- テスト環境セットアップ
 - ユニット / コンポーネント / E2E テスト
 - デプロイ前チェック
 - Vercel デプロイ
-

18. 付録：設定ファイルパス早見表

機能	ファイルパス	Git共有
ユーザー設定	<code>~/.claude/settings.json</code>	No
プロジェクト設定	<code>.claude/settings.json</code>	Yes
ローカル設定	<code>.claude/settings.local.json</code>	No
CLAUDE.md（共有）	<code>./CLAUDE.md</code> or <code>./.claude/CLAUDE.md</code>	Yes
CLAUDE.md（個人）	<code>./CLAUDE.local.md</code>	No
CLAUDE.md（全体）	<code>~/.claude/CLAUDE.md</code>	No
AGENTS.md（互換用）	<code>./AGENTS.md</code>	Yes
Rules	<code>.claude/rules/*.md</code>	Yes
Skills	<code>.claude/skills/<name>/SKILL.md</code>	Yes
Skills（個人）	<code>~/.claude/skills/<name>/SKILL.md</code>	No
Subagents	<code>.claude/agents/<name>.md</code>	Yes
Subagents（個人）	<code>~/.claude/agents/<name>.md</code>	No
Hooks	<code>settings.json</code> 内の <code>hooks</code>	設定場所による
MCP（共有）	<code>.claude/.mcp.json</code>	Yes
MCP（個人）	<code>.claude/.mcp.local.json</code>	No
MCP（全体）	<code>~/.claude/.mcp.json</code>	No
Keybindings	<code>~/.claude/keybindings.json</code>	No
Memory	<code>~/.claude/projects/<project>/memory/</code>	No
Worktree	<code>.claude/worktrees/<name>/</code>	No
Worktreeインクルード	<code>.worktreeinclude</code>	Yes

付録：開発開始から完了までのチェックリスト

=== Phase 0: 環境構築 ===

- Claude Code インストール済み
- settings.json で危険コマンドをブロック済み
- Permissions の allow/deny リスト設定済み
- MCP サーバー設定済み (GitHub, Supabase等)
- Hooks 設定済み (自動フォーマット、保護ファイル、通知)
- Keybindings カスタマイズ済み (必要なら)

=== Phase 1: プロジェクト初期化 ===

- Next.js プロジェクト作成済み
- 追加パッケージインストール済み
- Git リポジトリ初期化済み
- .claude/ ディレクトリ構造作成済み
- .gitignore 設定済み (.env, settings.local.json等)

=== Phase 2: 要件定義 (Plan Mode) ===

- アプリ全体像の定義完了
- 画面一覧と画面設計完了
- データベース設計完了
- API一覧と設計完了
- 設計ドキュメント保存済み (docs/)

=== Phase 3-6: 開発基盤 ===

- CLAUDE.md 作成済み
- CLAUDE.local.md 作成済み (必要なら)
- .claude/rules/ にルールファイル配置済み
- .claude/skills/ にSkill作成済み
- .claude/agents/ にSubagent定義済み
- ESLint 設定済み
- Prettier 設定済み
- TypeScript strict設定済み

=== Phase 7: 機能開発 ===

- 基盤 (レイアウト、DB接続、共通コンポーネント) 構築済み
- 認証機能 実装・レビュー・テスト・コミット済み
- 主要CRUD 実装・レビュー・テスト・コミット済み
- 高度な機能 実装・レビュー・テスト・コミット済み
- UI/UX改善済み

=== Phase 8-9: テスト・デプロイ ===

- テスト環境セットアップ済み
- ユニットテスト作成・パス済み
- コンポーネントテスト作成・パス済み
- E2Eテスト作成・パス済み
- ビルド成功確認済み (npm run build)
- 型チェック通過済み (npx tsc --noEmit)

- Lint通過済み (npm run lint)
- console.log / any型 / TODO 残りなし
- 環境変数整理済み (.env.example)
- npm audit 脆弱性なし
- デプロイ完了

最後に: バイブコーディングの核心は「AIに丸投げ」ではなく「AIと対話しながら品質の高いコードを段階的に構築する」ことです。

Claude Codeの全機能 (Settings、Hooks、Skills、Rules、MCP、Plan Mode、Worktrees、Subagents、Memory) を適切なフェーズで活用し、要件を明確にし、段階的に進め、各段階でレビューすることで、プロダクション品質のアプリケーションを効率的に開発できます。

特に重要な3つの習慣: 1. 「まだコードは書かないで」 — 方針合意を先にする 2. 「完了したら一度止まって」 — 段階的レビューで品質を維持 3. 「/review → /commit」 — レビューしてからコミットする

そして大前提として、`git status` で現状を確認してから依頼し、`git diff` で結果を確認してから commit、`git push` は人間が手動で行う、というGit運用を徹底することで、AIと安全に協調できます。